

Data Structures

CSC 301

Lecture Notes

Chapter 2: Pointers and Memory Management

Computer Science
Osun State University
2026/2027

Prepared by
Tunde Sardine

Contents

2	Pointers and Memory Management	2
2.1	Introduction	2
2.2	Pointers and References	2
2.2.1	Pointers	2
2.2.2	References	2
2.2.3	Dereferencing	3
2.3	Memory Allocation: Stack vs. Heap	3
2.3.1	Stack Allocation	3
2.3.2	Heap Allocation	3
2.4	Run-Time Storage Management	3
2.5	Conclusion	3

Chapter 2

Pointers and Memory Management

2.1 Introduction

This chapter explores the fundamental concepts of pointers and memory management, which are critical for understanding how data structures interact with computer memory. Pointers and references enable dynamic memory allocation and efficient data manipulation, while stack and heap allocation strategies determine how memory is organized during program execution. Run-time storage management ensures that memory is allocated and deallocated appropriately, preventing issues such as memory leaks or dangling pointers.

2.2 Pointers and References

2.2.1 Pointers

A **Pointer** is a variable that stores the memory address of another variable or data structure. Unlike direct references to data, pointers allow indirect access, enabling operations such as dynamic memory allocation and traversal of complex data structures. For example, in C, a pointer to an integer is declared as `int *ptr;`, and its value is set using the address-of operator `&`.

2.2.2 References

A **Reference** is a high-level language construct that provides an indirect way to access memory locations, similar to pointers. However, references are typically managed automatically by the language runtime, ensuring safety and preventing null references. In languages like C++, references are declared as `int ref = var;`, where `ref` is tied to the lifetime of `var`.

2.2.3 Dereferencing

Dereferencing a pointer involves accessing the value stored at the memory address it holds. This is achieved using the dereference operator `*`. For instance, if `ptr` points to an integer `x`, `*ptr` retrieves the value of `x`. Improper dereferencing can lead to runtime errors, such as accessing invalid memory.

2.3 Memory Allocation: Stack vs. Heap

2.3.1 Stack Allocation

Stack Allocation is a memory allocation strategy where data is stored in a last-in, first-out (LIFO) structure. It is typically used for local variables and function call management. Stack allocation is fast and automatic, as memory is released when the function exits. However, it has limited size and cannot be resized dynamically.

2.3.2 Heap Allocation

Heap Allocation involves dynamic memory allocation from a pool of memory called the heap. This allows for flexible memory usage, as data can be allocated and deallocated at runtime. Functions like `malloc()` in C or `new` in C++ are used to request memory on the heap. Unlike stack allocation, heap memory must be explicitly freed to avoid memory leaks.

2.4 Run-Time Storage Management

Run-time storage management encompasses the processes of allocating and deallocating memory during program execution. This includes manual management, where the programmer controls memory (e.g., using `free()` in C), and automatic management, such as garbage collection in languages like Java. Efficient run-time management is essential for optimizing performance and preventing memory-related errors.

2.5 Conclusion

Understanding pointers, memory allocation strategies, and run-time storage management is vital for designing efficient data structures. These concepts form the foundation for implementing complex algorithms and managing resources effectively in software development.

Tutorial Questions

1. Define "Pointers" using the terminology registry.
2. Compare stack and heap allocation in terms of memory management.

3. What is the difference between a pointer and a reference?
4. Explain the concept of memory leaks and how they can occur.
5. Write a C function that allocates memory on the heap and returns a pointer.
6. Discuss the implications of improper memory management.
7. How does garbage collection differ from manual memory management?
8. Provide an example of a data structure that uses linked structures.